15

Лекция 15–16 декабря

Коды, исправляющие ошибки

Константин Чухарев

Дискретная математика — Осень 2026

Коды, исправляющие ошибки

> *Цель вычислений — проникновение в суть, а не числа.*
>
> ***— Ричард Хэмминг***

Канал шумит

Канал передачи данных искажает биты. Ноль приходит как единица, единица как ноль.

Ошибки приходят и поодиночке, и пачками подряд.

Переспросить канал удаётся не всегда. Диск читают спустя годы, а память отвечает сразу, и повторной передачи не будет.

- В памяти с коррекцией бит переворачивается от космической частицы.
- Царапина на диске портит дорожку не по одному биту, а целым участком.

Слова и коды

Данные приходят словами над алфавитом, а код оставляет лишь разрешённые слова.

Определение 1 (Алфавит и слово)

Алфавит — конечное непустое множество символов.

Слово над Σ — конечная последовательность символов, а **длина** слова — число символов.

Определение 2 (Код)

Код — непустое множество слов, которые называются **кодовыми словами**.

Кодирование — инъективная функция $E : M \to C$ из сообщений в кодовые слова, а **декодирование** возвращает $D(E(m)) = m$.

Из инъективности следует $|M| \leq |C|$. Запас $|C| - |M|$ — будущая избыточность кода.

Кодирование без инъекции

Отображение сообщений в слова обязано быть инъективным, иначе декодер не различит два сообщения.

Пример

Кодирование трёх сообщений задано равенствами.

$$E(A) = 0, \quad E(B) = 1, \quad E(C) = 0.$$

Принятое слово 0 с равным успехом могло быть A или C.

Ответа нет, потому что различие между сообщениями стёрто ещё до канала.

Инъекция не заменяет избыточность.

Возьмём $E(A) = 0$ и $E(B) = 1$. Отображение инъективно, но один перевёрнутый бит переводит A в B, и подмена проходит незамеченной.

Чётность как первая защита

Простейшая избыточность — один бит, равный XOR данных.

Пример

К двум битам данных добавим третий, их XOR.

$$00 \mapsto 000, \quad 01 \mapsto 011, \quad 10 \mapsto 101, \quad 11 \mapsto 110.$$

Передали 00, а канал перевернул второй бит. Принято 010, и чётность нарушена, ошибку видно.

Но какой бит испорчен, не узнать, ведь от принятого слова три кодовых слова отстоят на один бит.

Две ошибки дают чётное число единиц. Из 00 выходит 110, снова кодовое слово.

Код чётности с $d = 2$ обнаруживает одну ошибку и не исправляет ни одной.

Код-повторение

Повторение тратит больше канала, зато исправляет ошибку, а не только замечает её.

Пример

Код-повторение передаёт каждый бит трижды, и словами служат 000 и 111.

Принято 011, в нём две единицы против одного нуля. Голосование по большинству возвращает 1.

Одна ошибка исправляется, но два из трёх битов канала потрачены на защиту, а не на данные.

Защита оплачивается скоростью.

Код-повторение тратит две трети канала, чтобы исправить один бит.

Расстояние Хэмминга

Ошибки описываются позициями, и для этого словам нужно расстояние.

Определение 3 (Расстояние Хэмминга)

Расстояние Хэмминга — число позиций, в которых строки различаются.

$$d(x, y) = |\{i \mid x_i \neq y_i\}|.$$

Пример

Слова 10110 и 10001 расходятся в позициях 3, 4 и 5, поэтому $d(10110, 10001) = 3$. Перевернуть нужно ровно три бита.

Расстояние Хэмминга — метрика.

$$d(x, y) \geq 0, \quad d(x, y) = d(y, x), \quad d(x, z) \leq d(x, y) + d(y, z).$$

Неравенство треугольника держится покоординатно, и хватает разбора одной позиции.

Вес слова

Вес слова — это его расстояние до нулевого слова.

Определение 4 (Вес слова)

Весом слова x называется число его единичных позиций, обозначаемое $\mathrm{wt}(x)$.

i	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
x	1	0	1	1	0	1	0	0
y	1	0	0	1	1	1	1	0
$x+y$	0	0	1	0	1	0	1	0

Единицы слова 10110 стоят в позициях 1, 3 и 4, отсюда вес 3.

В таблице разошлись позиции 3, 5 и 7, и в сумме три единицы, поэтому $d(x,y) = \mathrm{wt}(x+y)$. Позиции расхождения — ровно единицы XOR.

Кодовое расстояние

Расстояние между словами превращается в характеристику всего кода.

Определение 5 (Кодовое расстояние)

Кодовое расстояние — минимум расстояний между разными кодовыми словами.

$$d = \min(\{d(x, y) \mid x, y \in C, x \neq y\}).$$

Пример

В коде-повторении $\{000, 111\}$ пара слов всего одна, и она разошлась в трёх позициях, поэтому $d = 3$.

В коде чётности $\{000, 011, 101, 110\}$ слова 011 и 101 различаются в двух позициях, и ближе пары нет, поэтому $d = 2$.

Расстояние — свойство самого кода, а число ошибок приходит из канала.

Параметры и скорость

Код измеряется тремя числами, и по ним сравнивают разные коды.

Определение 6 (Параметры кода)

Параметрами кода называют тройку $(n, |C|, d)$ — длину, число слов и кодовое расстояние. Для линейных кодов пишут (n, k, d) и $|C| = 2^k$.

Скорость $R = \frac{k}{n}$ — доля информационных битов, а избыточность $1 - R$ — доля служебных.

Код	Параметры	Скорость	Исправляет
Повторение	$(3, 1, 3)$	$1/3 \approx 0.33$	1 бит
Хэмминг	$(7, 4, 3)$	$4/7 \approx 0.57$	1 бит

При одинаковой надёжности код Хэмминга передаёт примерно в $\frac{12}{7} \approx 1.7$ раза больше данных, чем повторение.

Ближайшее слово

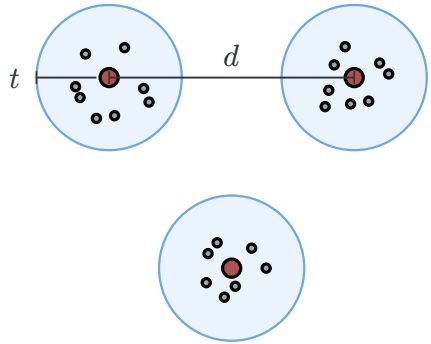
Кодовые слова — точки в пространстве всех двоичных строк длины n, и принятое слово r попадает туда же.

Декодер возвращает не то, что пришло из канала, а ближайшее кодовое слово, для которого $d(r, c)$ минимально.

При независимых ошибках с вероятностью $p < 1/2$ ближайшее слово совпадает с самым вероятным. Каждая перевёрнутая позиция умножает вероятность на p, каждая верная — на $1 - p$, а $p < 1 - p$.

Строка на равном расстоянии от двух кодовых слов не оставляет декодеру единственного выбора.

Шары вокруг кодовых слов



Шар радиуса t вокруг кодового слова — все строки, отличающиеся от него не более чем в t позициях.

Пока $2t + 1 \leq d$, шары попарно не пересекаются, и у принятой строки ровно один центр.

Объём шара растёт с t, поэтому шары занимают тем большую долю пространства, чем больше ошибок обещает код.

Способность кода

Расстояние переводится в число исправляемых ошибок одним неравенством.

Теорема 1 (Обнаружение и исправление)

Если $d \geq s + 1$, код обнаруживает до s ошибок.

Если $d \geq 2t + 1$, код исправляет до t ошибок, то есть

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor.$$

Пример

Код Хэмминга $(7, 4, 3)$ исправляет одну ошибку и замечает две.

Код с $d = 1$ не замечает ничего, ведь одна ошибка переводит слово в другое кодовое.

Код Хэмминга (7, 4)

Код Хэмминга укладывает четыре бита данных в семь бит канала и исправляет одну ошибку.

Определение 7 (Код Хэмминга)

Код Хэмминга имеет длину $n = 7$, несёт $k = 4$ бита данных и имеет расстояние $d = 3$.

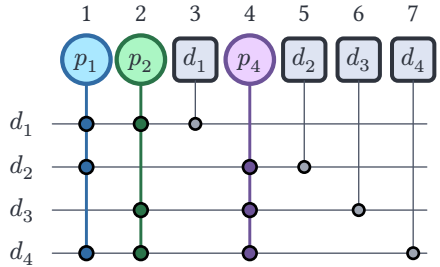
Проверочные биты занимают позиции $1, 2, 4$, то есть степени двойки, а данные стоят на позициях $3, 5, 6, 7$.

$$p_1, p_2, d_1, p_4, d_2, d_3, d_4.$$

Хэмминг работал на релейной машине, которая останавливалась при каждой ошибке ввода. Перезапуск обнулял вычисления, и в 1950 году он опубликовал первый явный код с исправлением ошибки.

Контрольные группы

Каждый проверочный бит — XOR данных своей группы, и группы выбраны по двоичным номерам позиций. Бит p_i контролирует те позиции, у которых i-й бит номера равен 1.



- p_1 контролирует d_1, d_2, d_4.
- p_2 контролирует d_1, d_3, d_4.
- p_4 контролирует d_2, d_3, d_4.

Позиция $3 = 011$ входит в первые две группы, позиция $7 = 111$ — во все три.

Кодирование

Собираем слово из данных и трёх проверок.

Пример

Данные $d_1 = 1, d_2 = 0, d_3 = 1, d_4 = 1$.

$$p_1 = 1 \oplus 0 \oplus 1 = 0, \quad p_2 = 1 \oplus 1 \oplus 1 = 1, \quad p_4 = 0 \oplus 1 \oplus 1 = 0.$$

Полученное кодовое слово равно $(0, 1, 1, 0, 0, 1, 1)$.

Каждая контрольная группа в слове 0110011 даёт нулевой XOR — слово кодовое.

Каждый проверочный бит — один XOR своей группы, поэтому три бита защиты обходятся в три операции над четырьмя битами данных.

Синдром

Приём повторяет те же три XOR, и их результат называется синдромом.

Определение 8 (Синдром)

Синдромом называют биты s_1, s_2, s_4, равные XOR своих контрольных групп.

$$s_1 = p_1 \oplus d_1 \oplus d_2 \oplus d_4,$$
$$s_2 = p_2 \oplus d_1 \oplus d_3 \oplus d_4,$$
$$s_4 = p_4 \oplus d_2 \oplus d_3 \oplus d_4.$$

Нулевой синдром означает, что ошибок не обнаружено.

s_1 — младший разряд номера, ведь p_1 стоит на позиции с номером 1, а s_4 — старший.

Ненулевой синдром читается как двоичное число $(s_4 s_2 s_1)_2$.

Оно равно номеру искажённой позиции, поэтому надо лишь перевернуть этот бит.

Исправление по синдрому

Проверим исправление на одном перевёрнутом бите.

Пример

Передано $(0, 1, 1, 0, 0, 1, 1)$, а ошибка в позиции 5 испортила слово до $(0, 1, 1, 0, 1, 1, 1)$.

$$s_1 = 0 \oplus 1 \oplus 1 \oplus 1 = 1, \quad s_2 = 1 \oplus 1 \oplus 1 \oplus 1 = 0, \quad s_4 = 0 \oplus 1 \oplus 1 \oplus 1 = 1.$$

Синдром $(1, 0, 1) = 5$ указывает на пятый бит, и мы его переворачиваем. Слово $(0, 1, 1, 0, 0, 1, 1)$ и данные $(1, 0, 1, 1)$ вернулись.

Кодирование и декодирование зеркальны, в них те же три группы и тот же XOR.

Линейные коды

Код Хэмминга замкнут относительно XOR, и замыкание позволяет считать коды линейной алгеброй.

Определение 9 (Линейный код)

Множество C двоичных слов длины n называется **линейным кодом**, если оно содержит нулевое слово и замкнуто относительно покоординатного XOR.

Пример

Код чётности $\{000, 011, 101, 110\}$ линейный. Сумма $011 + 101 = 110$ снова лежит в коде, и чётность суммы двух чётных слов сохраняется.

Базис из k строк порождает ровно 2^k слов, поэтому у линейного кода $|C| = 2^k$ и параметры пишутся (n, k, d).

Порождающая матрица

Кодировать сообщение удобно умножением на матрицу, строки которой задают базис кода.

Определение 10 (Порождающая матрица)

Матрица G размера $k \times n$, строки которой образуют базис кода, называется **порождающей**.

Кодирование сообщения m из k битов — умножение $c = mG$.

Разные подмножества строк дают разные суммы, поэтому 2^k сообщений получают 2^k разных слов.

Пример

Для кода чётности длины 4 строки G равны 1001, 0101 и 0011.

Сообщение $(1, 0, 1)$ складывает строки 1 и 3, и слово равно $1001 + 0011 = 1010$.

Код Хэмминга из базиса

Строки базиса — четыре кодовых слова, из которых складываются все остальные.

Пример

Порождающая матрица кода Хэмминга выглядит так.

$$G = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$$

Для сообщения $(1, 0, 1, 1)$ складываем строки 1, 3 и 4 и получаем $(0, 1, 1, 0, 0, 1, 1)$.

Первая, вторая и четвёртая координаты произведения — проверочные биты p_1, p_2, p_4 из трёх контрольных групп.

Поле из двух элементов

Множество двоичных строк длины n — векторное пространство $\mathrm{GF}(2)^n$, а линейный код в нём — подпространство.

Определение 11 (Поле $\mathrm{GF}(2)$)

Поле $\mathrm{GF}(2)$ — это 0 и 1, сложение — XOR, умножение — AND, $1 + 1 = 0$.

$\mathrm{GF}(2)^n$ — векторное пространство строк, а не поле из 2^n элементов.

Поле $\mathrm{GF}(2^n)$ строят как многочлены по модулю неприводимого, и оно нужно для байтовых каналов.

Пример

Код чётности в $\mathrm{GF}(2)^3$ содержит четыре слова, поэтому $k = 2$, а расстояние $d = 2$ даёт параметры $(3, 2, 2)$.

Доказательство способности кода

Оба утверждения следуют из оценки расстояний.

Доказательство

Обнаружение. Если $d \geq s + 1$ и ошибок не больше s, принятое слово отстоит от переданного меньше чем на d и кодовым не является.

Исправление. Пусть $d \geq 2t + 1$ и шары радиуса t вокруг кодовых слов. Если бы строка z лежала в двух шарах с центрами x и y, неравенство треугольника дало бы

$$d(x, y) \leq d(x, z) + d(z, y) \leq 2t,$$

а это противоречит условию.

Значит, принятое слово с не более чем t ошибками попадает ровно в один шар.

Почему не $\left\lfloor \frac{d}{2} \right\rfloor$?

Проверим формулу на коде, где половина расстояния достижима.

Считать способность кода по формуле $t = \left\lfloor \frac{d}{2} \right\rfloor$ — типичная ошибка.

Код $\{0000, 1111\}$ имеет $d = 4$, и формула обещает $t = 2$.

Но $d(0011, 0000) = 2$ и $d(0011, 1111) = 2$, поэтому декодеру нечего выбрать.

Верный счёт даёт $t = \left\lfloor \frac{d-1}{2} \right\rfloor = 1$.

Половина расстояния оставляет пограничное слово между шарами, и его нельзя отнести ни к одному центру.

Проверочная матрица

Три контрольные группы собираются в одну матрицу, и проверка слова становится умножением.

Определение 12

Проверочная матрица кода — матрица H размера $(n-k) \times n$, строки которой задают контрольные группы.

Код состоит в точности из решений уравнения $Hc^T = 0$ над $\mathrm{GF}(2)$.

$$H = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

Столбец j матрицы — двоичная запись числа j, если читать биты снизу вверх, от младшего к старшему. Семь столбцов исчерпывают все ненулевые тройки битов.

Проверка словом

Умножение на H отличает кодовое слово от испорченного.

Пример

Для кодового слова $(0, 1, 1, 0, 0, 1, 1)$ произведение Hx^T равно $(0, 0, 0)^T$.

Для принятого $(0, 1, 1, 0, 0, 1, 0)$ оно равно $(1, 1, 1)^T$, а это седьмой столбец H, то есть номер ошибки.

Для другой длины набор столбцов приходится выбирать. При $n = 2^l - 1$ берут все ненулевые столбцы высоты l, и проверочных битов выходит ровно l.

Синдром зависит только от ошибки

Кодовое слово даёт нулевой синдром, поэтому в синдроме виден только вектор ошибок.

$$Hr^T = H(c+e)^T = Hc^T + He^T = He^T.$$

Произведение He^T — XOR столбцов тех позиций, где у e стоит единица. Одна ошибка даёт столбец своей позиции, две ошибки — сумму двух столбцов.

Пример

Испорчены позиции 4 и 6. Их столбцы дают $(0,0,1) + (0,1,1) = (0,1,0)$, то есть второй столбец.

Синдром 2 называет позицию, где ошибки нет.

Сумма двух столбцов снова ненулевая и снова столбец H, поэтому две ошибки неотличимы от одной.

Расстояние линейного кода

Перебор пар заменяется перебором слов, ведь разность кодовых слов снова лежит в коде.

Теорема 2 (Расстояние и вес)

Кодовое расстояние линейного кода равно минимальному весу ненулевого слова,

$$d = \min(\{\mathrm{wt}(c) \mid c \in C, c \neq 0\}).$$

Доказательство

Докажем двумя неравенствами.

Пусть c — ненулевое слово наименьшего веса. Нулевое слово принадлежит коду, и пара c и 0 даёт $d \leq \mathrm{wt}(c)$.

Обратно, для разных $x, y \in C$ сумма $x + y$ — ненулевое кодовое слово веса $d(x, y)$. Поэтому d не меньше веса наименьшего ненулевого слова.

Веса слов кода Хэмминга

У кода Хэмминга 120 пар заменяются 15 ненулевыми словами.

Теорема 3 (Минимальный вес)

Кодовое расстояние кода Хэмминга $(7, 4)$ равно 3.

Доказательство

Докажем перебором весов.

Код линейный, поэтому d — минимальный вес ненулевого слова.

Вес 1 оставил бы в Hx^T один столбец, а все семь столбцов ненулевые. Вес 2 невозможен, ведь столбцы различны и их сумма ненулевая.

Вес 3 достижим, ведь столбцы 1, 2 и 3 в сумме дают нулевой столбец, и слово 1110000 кодовое.

Две ошибки

Посмотрим, что делает декодер, когда ошибок больше одной.

Пример

Передано $(0,1,1,0,0,1,1)$, а ошибки в позициях 2 и 4 дали принятое слово $(0,0,1,1,0,1,1)$.

Столбцы 2 и 4 дают $(0,1,0)+(0,0,1)=(0,1,1)$, то есть синдром 6.

Декодер переворачивает бит 6, а это бит данных d_3.

Исправленное слово — кодовое, и данные выходят $(1,0,0,1)$ вместо $(1,0,1,1)$.

Две ошибки маскируются под одну.

Код с $d=3$ их замечает, но исправить не может.

Три ошибки

Худший случай — когда сам вектор ошибок оказался кодовым словом.

Пример

Ошибки стоят в позициях 1, 2 и 3, и вектор ошибок $(1, 1, 1, 0, 0, 0, 0)$ сам является кодовым словом.

Принятое слово $(1, 0, 0, 0, 0, 1, 1)$ — снова кодовое слово. Синдром нулевой, поэтому декодер сообщает «ошибок нет», хотя данные выходят $(0, 0, 1, 1)$ вместо $(1, 0, 1, 1)$.

За пределами $t = \left\lfloor \frac{d-1}{2} \right\rfloor$ исправлений код за результат не отвечает.

Расширенный код (8, 4, 4)

К базовому коду добавляется один общий бит чётности всего слова.

Слова базового кода различаются минимум в трёх позициях, а при трёх различиях биты чётности тоже различны, поэтому расстояние равно 4.

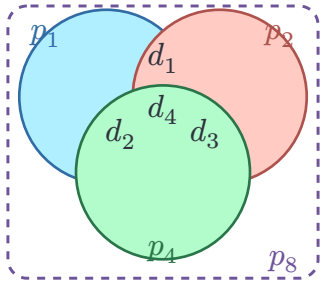
Теорема 4 (Общий бит чётности)

Код $(8, 4, 4)$ по-прежнему исправляет одну ошибку.

Зато он отличает двойную ошибку от одиночной, потому что двойная сохраняет чётность слова, а одиночная — нет.

Слово 00000000 с ошибками в позициях 1 и 2 читается как 11000000, и чётность сходится.

Круги проверок



Три круга — контрольные группы p_1, p_2, p_4. Пересечения кругов дают информационные биты $d_1, \ldots, d_4$, а внешняя рамка — общий бит чётности p_8.

Четыре исхода декодера

Синдром базового кода вместе с общим битом чётности различает четыре ситуации.

Синдром	Чётность	Что делать
0	сошлась	ошибок нет
0	не сошлась	испорчен бит чётности — исправляем
не 0	не сошлась	одна ошибка — исправляем
не 0	сошлась	две ошибки — не трогаем

Пример

Передано $(0,1,1,0,0,1,1,0)$ с данными $(1,0,1,1)$, а ошибки в позициях 4 и 6 дают $(0,1,1,1,0,0,1,0)$.

Синдром первых семи бит $(0,1,0)$, единиц в слове четыре, поэтому чётность сошлась, и декодер объявляет слово повреждённым.

Граница Хэмминга

Упаковка шаров ограничивает число кодовых слов сверху.

Теорема 5 (Граница Хэмминга)

Двоичный код длины n, исправляющий t ошибок, удовлетворяет неравенству

$$|C| \le \frac{2^n}{\sum_{i=0}^{t} \binom{n}{i}}.$$

Граница связывает число слов с объёмом шара, то есть с числом исправляемых ошибок.

Пример (Граница в числах)

При $n = 7$ и $t = 1$ шар вмещает $1 + 7 = 8$ строк, и граница даёт $|C| \le \frac{128}{8} = 16$.

Код $(7, 5, 3)$ из 32 слов запрещён, ведь $2^5 \cdot 8 = 256 > 128$.

Доказательство границы Хэмминга

Считаем точки в шарах и складываем их объёмы.

Доказательство

Докажем подсчётом строк в шарах.

Код исправляет t ошибок, поэтому $d \geq 2t + 1$ и шары радиуса t попарно не пересекаются.

На расстоянии ровно i от данного слова лежит $\binom{n}{i}$ строк, ведь искажённые позиции можно выбрать столькими способами. Значит, шар радиуса t вмещает ровно $\sum_{i=0}^{t} \binom{n}{i}$ строк.

Шары лежат внутри пространства из 2^n строк, поэтому

$$|C| \cdot \sum_{i=0}^{t} \binom{n}{i} \leq 2^n.$$

Совершенный код

Иногда шары заполняют пространство целиком.

Определение 13 (Плотная упаковка)

Код называется **совершенным**, если шары радиуса t вокруг его слов покрывают всё пространство без пересечений.

Для него граница Хэмминга обращается в равенство.

Пример

Код Хэмминга $(7, 4)$ исправляет одну ошибку, и

$$2^4 \cdot \left(1 + \binom{7}{1}\right) = 16 \cdot 8 = 128 = 2^7.$$

Код-повторение $(3, 1, 3)$ совершенный по той же причине, ведь $2 \cdot (1 + 3) = 8 = 2^3$.

Граница Синглтона

Второй предел приходит не из упаковки шаров, а из числа координат.

Теорема 6 (Граница Синглтона)

Код длины n с 2^k словами и кодовым расстоянием d удовлетворяет неравенству

$$k \leq n - d + 1.$$

Доказательство

Докажем вычёркиванием координат.

Вычеркнем из всех кодовых слов одни и те же $d-1$ позиций. Оставшиеся отрезки длины $n-d+1$ все различны, иначе два слова расходились бы не более чем в $d-1$ позициях. Отрезков такой длины ровно 2^{n-d+1}, поэтому $2^k \leq 2^{n-d+1}$.

Для кода Хэмминга $(7, 4, 3)$ правая часть равна 5, и неравенство выполнено с запасом.

Коды MDS

Граница Синглтона ставит вопрос, какие коды её достигают.

Определение 14 (Код MDS)

Код называется **кодом MDS** (от англ. maximum distance separable), если он достигает границы Синглтона, то есть $d = n - k + 1$.

Посчитаем $n - k + 1$ для знакомых кодов и сравним с d.

Код	(n, k, d)	$n - k + 1$	**MDS?**
Повторение	$(3, 1, 3)$	3	да
Чётность	$(n, n-1, 2)$	2	да
Хэмминг	$(7, 4, 3)$	4	нет
Расширенный Хэмминг	$(8, 4, 4)$	5	нет

Почему код $(7, 4, 4)$ невозможен?

Расширенный код получил расстояние 4 добавлением восьмого бита. Без него, на длине 7, расстояние 4 недостижимо, хотя граница Синглтона его разрешает.

Доказательство

Докажем от противного.

Пусть код с расстоянием 4 существует. Шары радиуса 1 вокруг его слов не пересекаются, в каждом по 8 строк, и

$$16 \cdot 8 = 128 = 2^7,$$

значит шары покрывают всё пространство.

Возьмём слово c и строку z на расстоянии 2 от него. Строка z лежит в чьём-то шаре, поэтому $d(c, c') \leq 3$ для другого слова c', вопреки расстоянию 4.

Код Хэмминга на Rust

Кодер — это данные на своих местах и три XOR по строкам матрицы H.

```
const H: [[u8; 7]; 3] = [
    [1, 0, 1, 0, 1, 0, 1],
    [0, 1, 1, 0, 0, 1, 1],
    [0, 0, 0, 1, 1, 1, 1],
];
const PARITY: [usize; 3] = [0, 1, 3]; // позиции 1, 2, 4

fn encode(data: [u8; 4]) -> [u8; 7] {
    let mut x = [0u8; 7];
    x[2] = data[0];
    x[4] = data[1];
    x[5] = data[2];
    x[6] = data[3];
```

Код Хэмминга на Rust [2]

```
    for (i, row) in PARITY.iter().zip(H.iter()) {
        x[*i] = (0..7)
            .filter(|&j| row[j] == 1 && j != *i)
            .map(|j| x[j])
            .sum::<u8>() % 2;
    }
    x
}
```

Декодер

Декодер считает синдром, читает в нём номер позиции и переворачивает бит.

```
fn decode(mut r: [u8; 7]) -> [u8; 4] {
    let mut s = [0u8; 3];
    for i in 0..3 {
        s[i] = (0..7).map(|j| H[i][j] * r[j]).sum::<u8>() % 2;
    }

    if s.iter().any(|&v| v != 0) {
        let pos = s[0] as usize + 2 * s[1] as usize + 4 * s[2] as usize;
        println!("синдром {:?}, ошибка в позиции {}", s, pos);
        r[pos - 1] ^= 1;
    }
    [r[2], r[4], r[5], r[6]]
}
```

Запуск с ошибкой

Шум моделируем одним XOR после кодирования.

```
let mut x = encode([1, 0, 1, 1]);
x[4] ^= 1; // канал перевернул пятый бит
println!("{:?}", decode(x));
```

```
синдром [1, 0, 1], ошибка в позиции 5
[1, 0, 1, 1]
```

Синдром $(1, 0, 1)$ прочитался как число 5, перевёрнутый бит встал на место, данные вернулись.

CRC и контрольные суммы

Обнаруживающие механизмы вокруг нас решают более простую задачу.

- Контрольная сумма пакета складывает его слова, и несовпадение сигнализирует о порче.
- CRC делит многочлен данных на порождающий многочлен, и нулевой остаток означает целостность.
- Ни контрольная сумма, ни CRC не знают позицию ошибки, поэтому исправить её не могут.

Обнаружение требует $d \geq s + 1$, а исправление — $d \geq 2t + 1$, поэтому коррекция вдвое дороже.

В сетях ставят CRC и просят повторную передачу, а коды Хэмминга живут в памяти, где переспроса нет.

Вопросы по кодам

Вопросы

1. Вычислите $d(110010, 010101)$.
2. Почему код чётности не исправляет ни одной ошибки, хотя обнаруживает одну?
3. Закодируйте $(0, 1, 1, 0)$, внесите одиночную ошибку и исправьте её по синдрому.
4. Почему линейному коду достаточно минимального веса ненулевого слова?
5. Код чётности $(n, n-1, 2)$ достигает границы Синглтона. Как это проверить?
6. Функция кодирования отправляет сообщения A и B в одно слово. Что нарушено и чем это грозит?
7. Дан код $\{000, 111\}$. Найдите его параметры $(n, |C|, d)$, скорость и проверьте границу Хэмминга.

Что почитать?

Задача	Где читать
Понять расстояние	m13, раздел «Расстояние Хэмминга и геометрия кода»
Построить код Хэмминга	m13, разделы «Код Хэмминга $(7, 4)$» и «Расширенный код Хэмминга $(8, 4, 4)$»
Разобрать линейные коды	m13, разделы «Линейные коды и $\mathrm{GF}(2)$» и «Конечные поля для кодов»
Выйти за двоичный алфавит	m13, раздел «Коды Рида–Соломона»
Увидеть границы	m13, разделы «Коды и комбинаторика» и «Граница Шеннона»
Проверить целостность	m13, раздел «Циклические коды»